Module 1 Activities – Boolean logic, binary and programming

i would convert 27 to binary by using the below since i find the class example too hard to follow personally

in binary, from the right to left

1 is 1s column

left is 2s columns

left of this is 4s columns, etc

so 1 1 0 1 1 is 27 in base 10

my calculation right to left = 1+ 2 + 0 + 8 + 16 = 27

Represent 110101_2 in decimal notation. Write out the steps you took to do the conversion.

1 + 0 + 4 + 0 + 16 + 32 = 53

Case study: Training heart rate

The Australian College of Sports Medicine advises maintaining your Training Heart Rate (THR) during an aerobic workout. The THR is calculated using the formula: $0.7 \times (220 - a) + 0.3 \times r$, where '$a$' represents your age and 'r' denotes your Resting Heart Rate (RHR), which is your pulse when you first wake up.

However, if you're not actively engaged in sports, the THR equation changes to: $0.8 \times (225 - a) + 0.35 \times r$.

We have a client who is 33 years old with a resting heart rate of 55 bpm. She's interested in knowing her THR. However, she prefers not to disclose whether she regularly participates in sports. Therefore, we need to provide her with both possible outcomes.

Step 3: Apply the business rules (based on the 5-step problem-solving process) to the case study.

1. **Identify** your inputs.

Inputs are age (33), resting heart rate (55) and the formula (THR equation)

2. **Identify** the goal or objective (related to the output).

The goal is to calculate the client's THR. As we don't know how active she is, we want to calculate a THR for both formulas. So we are aiming for two answers to give to the client.

3. **Create** a list of tasks to achieve your objective.

- Ask for client's age, resting heart rate and activity level
- Calculate client THR if regularly engaged in sports OR Calculate THR if not regularly engaged in sports
- Provide calculation to client

4. **Develop** a hand-written solution.

- Ask client for their age
- Ask client for resting heart rate
- Ask client for activity level
- If activate, calculate THR using 0.7 x (220 - (age)) + 0.3 x (RHR)
- Else if not active, calculate THR using 0.8 x (225 - (age)) + 0.35 x (RHR)
- Print answer to client with recommended THR for activity level

5. **Assess** (test) your solution.

Assumptions:

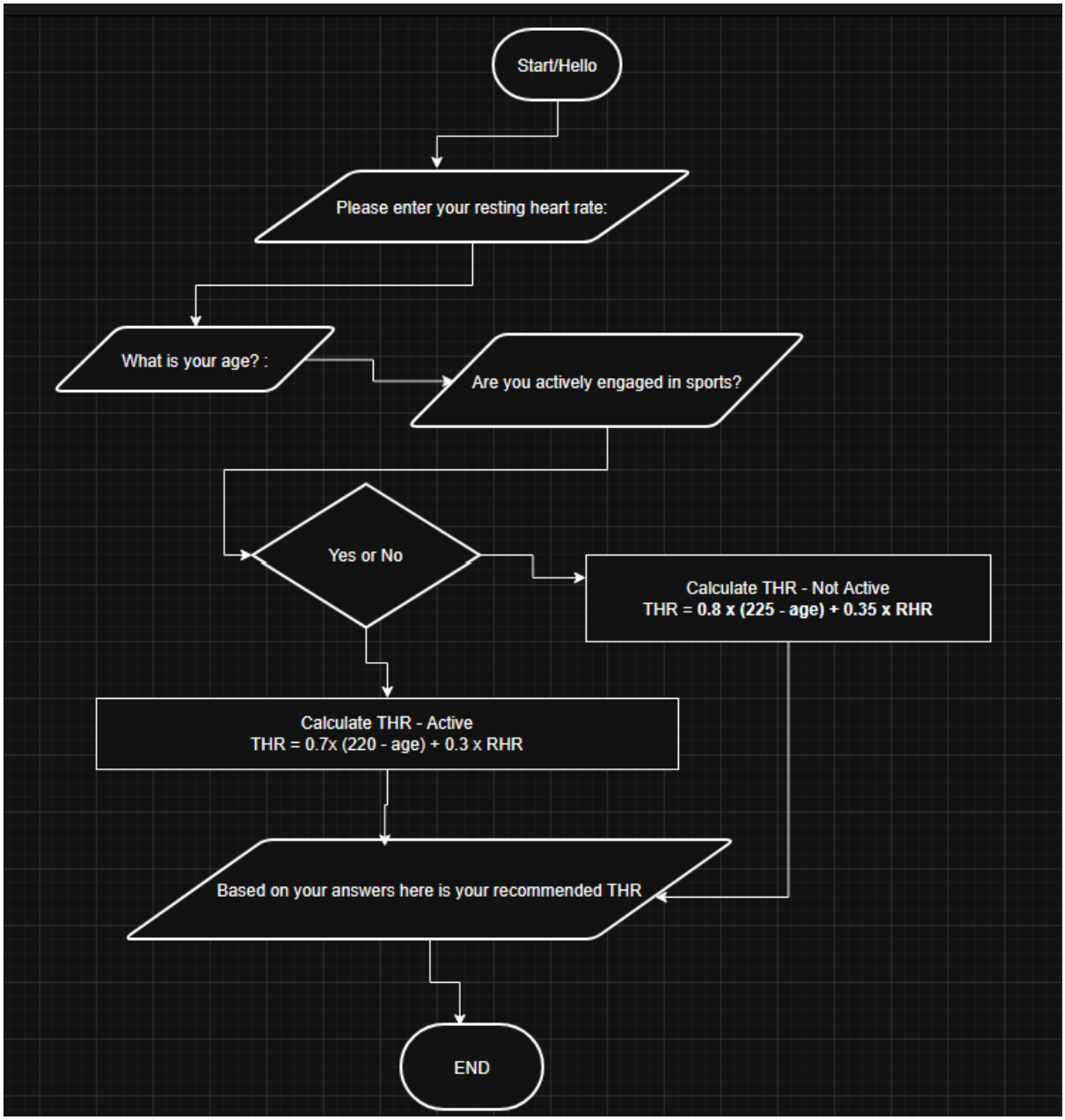
1. Age and RHR
2. Activity level
3. Formulas: I.E **0.8 x (225 - 33) + 0.35 x 55 (not active) and** 0.7 x (220 - 33) + 0.3 x 55 (active)

Test:

1. If active, use formula for actively enagaged in sports using inputs given
2. If not active, use formula for not actively engaged in sports using inputs given

Step 6: Code your solution.

1. **Produce** a flowchart using the flowchart template to describe the flow of instructions.



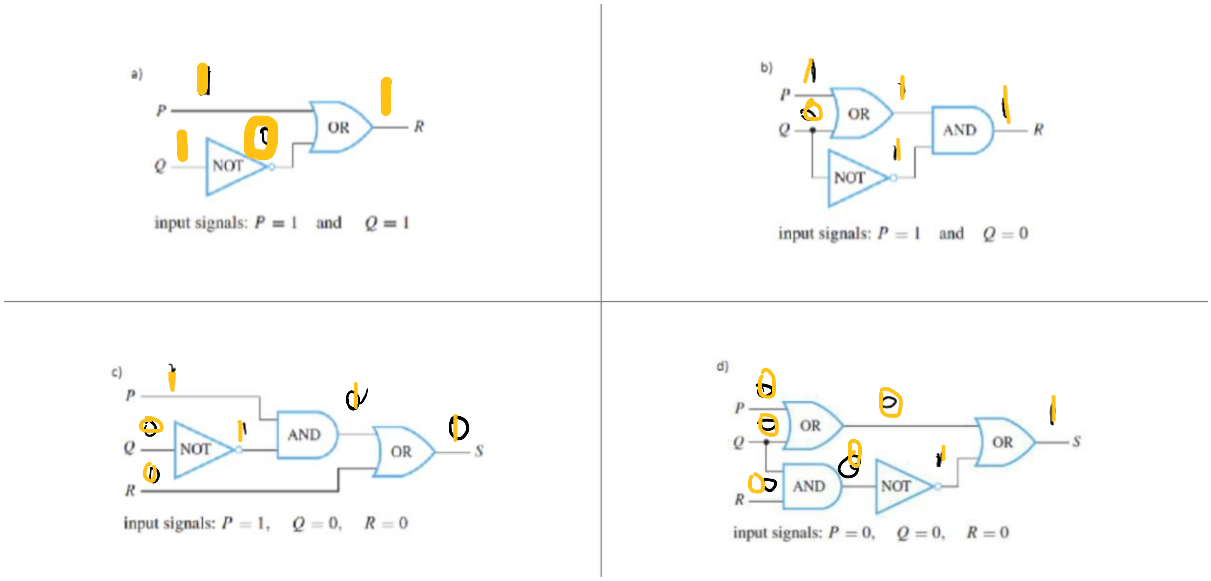
2. **Derive** a pseudocode following Step 3.

- Ask for inputs.
- "What is your age?"
- Receive integer
- "What is your resting heart rate (RHR)?"
- Receive integer
- "Are you actively engaged in sports?"
- Yes or No/True or False

- If yes, Calculate **the** Training Heart Rate **(**THR) using 0.7 x (220 - (age)) + 0.3 x (RHR), for regularly engaged in sports
- If n**o, Calculate the THR using 0.8 x (225 - (age)) + 0.35 x (RHR), for not regularly engaged in sports**
- Print calculated result
- End

Part 1: Logic gates

Step 1: Review the input signals of the following circuits and **answer** the question: What are the output signals for each of the circuits?



Step 2: Write an input/output table for each of the parts a) - d) in Step 1.

a)

Input	Input	NOT gate	Output
P	Q		R
1	1	0	1

b)

Input	Input	NOT gate	Output
P	Q		R
1	0	1	1

c)

Input	Input	Input	NOT gate	Output
P	Q	R		S
0	0	0	1	1

d)

Input	Input	Input	NOT gate	Output
P	Q	R		S
0	0	0	1	1

Step 3: Find the Boolean expression that corresponds to the circuit for each of the parts a) - d) in Step 1.

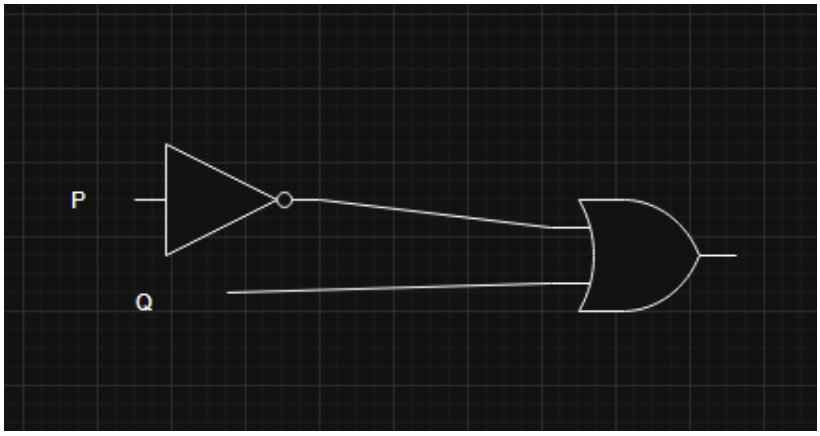
a) P v ~Q = R
b) (P v Q) ⋀ ~Q = R
c) $(P \bigwedge \sim Q) \text{ v } R = S$
d) $(P \text{ v } Q) \text{ v } \sim (Q \bigwedge R) = S$

Step 4: Construct circuits for the Boolean expressions indicated below.

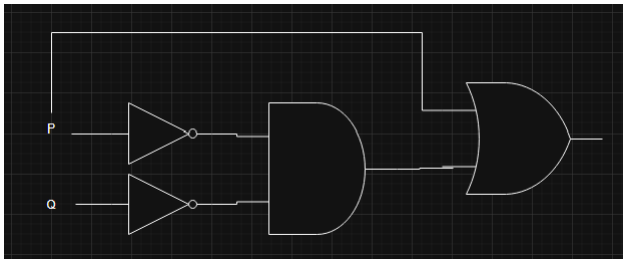
A. *~P ∨ Q*

B. *~P ∨ Q*

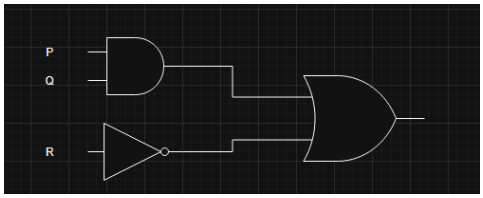
A and B were the same.



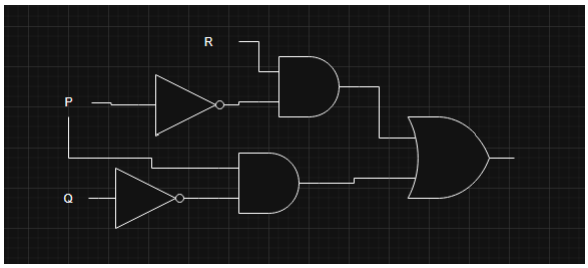
C. P ∨ (~P ∧ ~Q)



D. *(P ∧ Q) ∨ ~R*



E. (P ∧ ~Q) ∨ (~P ∧ R)



Step 5: Construct the following for each table.

a)

P	*Q*	*R*	*S*
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	0
0	0	0	0

b)

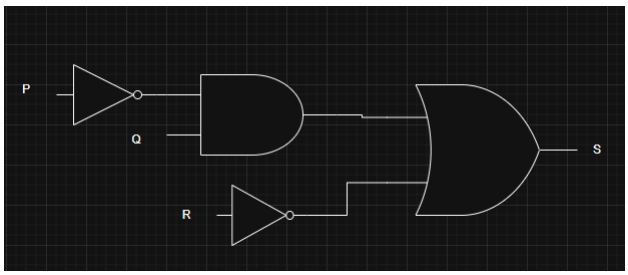
P	*Q*	*R*	*S*
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	0
0	0	0	0

- A Boolean expression having the given table as its truth table.

Just having a guess

a) (~P ∧ Q) v ~R = S

- A circuit having the given table as its input/output table.



Part 2: Numbering systems

Step 1: Represent the following decimal integers in binary notation.

a. 19 - 10011

b. 55 - 110111

c. 287 - 100011111

d. 458 - 111001010

e. 1609 - 11001001001

f. 1424 - 10110010000

Step 2: Represent the following binary numbers in decimal notation.

a. 1110 - 14

b. 10111 - 23

c. 110110 - 54

d. 1100101 - 101

e. 1000111 - 71

f. 1011011 – 91

Step 3: Perform the arithmetic given below using binary notation.

Note: The way we represent negative numbers above is a general, mathematical standard of representing negative numbers in any numbering system. We simply add a minus sign at the front. However, this is not the manner negative numbers are represented in a computer. The binary number representation in a computer only contains the value (or magnitude) of a value, not its sign, so techniques needed to be devised to allow storing both the sign and the value of a number. Various ways exist, in the following sections we will explore one's complement and two's complement.

a)	b)	c)	d)
1001_2 $+\ 101_2$	1001_2 $+\ 1011_2$	101101_2 $+\ 11101_2$	110111011_2 $+1001011010_2$
e)	**f)**	**g)**	**h)**
10100_2 $-\ 1101_2$	11010_2 $-\ 1101_2$	101101_2 $-\ 10011_2$	1010100_2 $-\ 10111_2$

A) 1110
B) 10100
C) 1001010
D) 10000010101
E) 100001
F) 100111
G) 1000000
H) 1101011

Part 3: One's complement

Ones' Complement overview

The Ones' Complement is a method used in binary arithmetic. In this system, positive numbers are represented in the same way as in the sign-and-magnitude method. The left-most bit is 0 for positive numbers. For negative numbers, the left-most bit is 1, indicating a negative sign.

Additionally, each bit in the binary representation of the number is "flipped". This means that a 0 is turned into a 1, and a 1 is turned into a 0. This method allows for the representation of both positive and negative numbers in binary form.

Example
Review the decimal number -124 using the Ones' Complement method and a 16-bit representation.

1. **Decimal: -124_{10}**
2. **Binary: $-111\ 1100_2$**
3. **16-bit binary representation of the absolute value (124_{10}):**
   $0000\ 0000\ 0111\ 1100_2$
4. **Flip the bits**
   $1111\ 1111\ 1000\ 0011_2$

Step 1: Review the following information.
Step 2: Write the one's complement of -9910.

If 9910 in binary is 0010011010110110

Then -9910 = 1101100101001001

Part 4: Two's complement

Two's Complement overview

In the Two's Complement system, the first bit of a binary number is used to indicate the sign: a 0 means the number is positive, while a 1 means it's negative.

The rest of the bits represent the absolute value of the number.

If the number is positive, we don't need to do anything else. But if it's negative, we need to convert it. We do this by flipping all the bits (turning 0s into 1s and vice versa), and then adding 1 to the result. This process gives us the Two's Complement of the number, which is a clever way to represent both positive and negative numbers using a fixed number of bits.

Step 1: Review the following information and an example.

Example

Review the decimal number -124 using the Two's Complement method and a 16-bit representation.

1. Decimal: -124_{10}
2. Binary: $-111\ 1100_2$
3. 16-bit binary representation of the absolute value (124_{10}):
   $0000\ 0000\ 0111\ 1100_2$

4. Flip the bits
   $1111\ 1111\ 1000\ 0011_2$
5. Add 1:
   $1111\ 1111\ 1000\ 0100_2$

Step 2: Convert the numbers +52 and -52 into the Two's Complement, using a 16-bit representation.

+52 = 00000 00000 110100

-52 = 11111 11111 001011 + 1 = 11111 11111 001100